```python
"""Annual PCU-demand scenarios for the supplied Metro Manila road dataset."""

from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

CLASSES = ("Car", "PUJ", "UV", "Taxi", "PUB", "Truck", "Trailer", "MC", "Tricycle")
FEATURES = ["Road_PCU_Volume", "Capacity_Limit", "Year", "Road_Type"] + [f"Share_{c}" for c in CLASSES]
RISK_ORDER = ["Low", "Medium", "High"]
RING_ROADS = ("C-4_EDSA", "C-5_CP_Garcia")
PREDEFINED_SCENARIOS = (
    ("Baseline 4.5%", {"growth_rate": 0.045}),
    ("Accelerated 7%", {"growth_rate": 0.07}),
    ("Transit shift 20%", {"growth_rate": 0.045, "transfer_share": 0.20}),
    ("EDSA/C-5 expansion 20%", {"growth_rate": 0.045, "capacity_increase": 0.20}),
)


def risk_from_ratio(ratio):
    values = np.asarray(ratio, dtype=float)
    labels = np.select([values >= 0.95, values >= 0.70], ["High", "Medium"], default="Low")
    return str(labels.item()) if labels.ndim == 0 else labels


def validate_data(df):
    required = {"Year", "Road_ID_Name", "Road_Type", "Capacity_Limit_PCU_Per_Day", "Road_PCU_Volume_Per_Day"}
    required |= set(CLASSES) | {f"PCEF_{c}" for c in CLASSES}
    missing = sorted(required - set(df.columns))
    if missing:
        raise ValueError(f"Missing columns: {missing}")
    data = df.copy()
    numeric = list(CLASSES) + [f"PCEF_{c}" for c in CLASSES] + ["Year", "Capacity_Limit_PCU_Per_Day", "Road_PCU_Volume_Per_Day"]
    for col in numeric:
        data[col] = pd.to_numeric(data[col], errors="raise")
    if data[numeric].isna().any().any():
        raise ValueError("Missing numeric values")
    if data.duplicated(["Road_ID_Name", "Year"]).any():
        raise ValueError("Duplicate road-year rows")
    if len(data) != 120 or data.Road_ID_Name.nunique() != 20 or set(data.Year) != set(range(2020, 2026)):
        raise ValueError("Expected 20 roads for every year 2020?2025")
    if data.groupby("Road_ID_Name").Year.nunique().ne(6).any():
        raise ValueError("A road is missing a historical year")
    if (data["Capacity_Limit_PCU_Per_Day"] <= 0).any() or (data[list(CLASSES)] < 0).any().any():
        raise ValueError("Capacity must be positive and counts nonnegative")
    return data.sort_values(["Road_ID_Name", "Year"]).reset_index(drop=True)


def add_pcu_and_risk(df):
```

```
    data = df.copy()
    data["Road_PCU_Volume"] = sum(data[c] * data[f"PCEF_{c}"] for c in CLASSES)
    data["Capacity_Limit"] = data["Capacity_Limit_PCU_Per_Day"]
    data["PCU_Difference"] = data["Road_PCU_Volume"] - data["Road_PCU_Volume_Per_Day"]
    raw_total = data[list(CLASSES)].sum(axis=1)
    for c in CLASSES:
        data[f"Share_{c}"] = np.where(raw_total > 0, data[c] / raw_total, 0.0)
    data["VC_Ratio"] = data["Road_PCU_Volume"] / data["Capacity_Limit"]
    data["Rule_Risk_Level"] = risk_from_ratio(data["VC_Ratio"])
    return data


def make_features(df):
    return df[FEATURES]


def fit_and_evaluate(history):
    train = history[history.Year <= 2023]
    test = history[history.Year >= 2024]
    prep = ColumnTransformer([
        ("road_type", OneHotEncoder(handle_unknown="ignore"), ["Road_Type"]),
        ("numeric", "passthrough", [c for c in FEATURES if c != "Road_Type"]),
    ])
    model = Pipeline([("features", prep), ("forest", RandomForestClassifier(n_estimators=300, max_depth=8, min_samples_leaf=2, random_state=42, class_weight="balanced"))])
    model.fit(make_features(train), train.Rule_Risk_Level)
    pred = model.predict(make_features(test))
    report = classification_report(test.Rule_Risk_Level, pred, labels=RISK_ORDER, output_dict=True, zero_division=0)
    metrics = pd.DataFrame(report).T
    matrix = pd.DataFrame(confusion_matrix(test.Rule_Risk_Level, pred, labels=RISK_ORDER), index=RISK_ORDER, columns=RISK_ORDER)
    comparison = test[["Year", "Road_ID_Name", "Road_Type", "VC_Ratio", "Rule_Risk_Level"]].copy()
    comparison["RF_Risk_Level"] = pred
    comparison["Disagreement"] = comparison.Rule_Risk_Level.ne(comparison.RF_Risk_Level)
    return model, metrics, matrix, comparison


def simulate(history, model, name, growth_rate=0.045, transfer_share=0.0, capacity_increase=0.0, expansion_year=2028):
    """Transfer share moves car *people* to buses: one new bus per 40 shifted cars."""
    if not 0 <= transfer_share <= 1 or growth_rate < -1 or capacity_increase < 0:
        raise ValueError("Invalid scenario parameters")
    baseline = history.loc[history.Year == 2025].copy()
    output = []
    for year in range(2025, 2036):
        state = baseline.copy()
        state["Year"] = year
        factor = (1 + growth_rate) ** (year - 2025)
        for c in CLASSES:
            state[c] = baseline[c].to_numpy(dtype=float) * factor
        if year > 2025 and transfer_share:
            shifted_cars = state.Car * transfer_share
            state["Car"] -= shifted_cars
            state["PUB"] += shifted_cars / 40.0
        state["Capacity_Limit_PCU_Per_Day"] = baseline.Capacity_Limit.to_numpy(dtype=float)
        if year >= expansion_year and capacity_increase:
```

```
            mask = state.Road_ID_Name.isin(RING_ROADS)
            state.loc[mask, "Capacity_Limit_PCU_Per_Day"] *= 1 + capacity_increase
        state = add_pcu_and_risk(state)
        state["RF_Risk_Level"] = model.predict(make_features(state))
        state["Risk_Level"] = state["Rule_Risk_Level"]
        state["RF_Disagreement"] = state.RF_Risk_Level.ne(state.Risk_Level)
        state["Scenario"] = name
        state["Capacity_Basis"] = "Synthetic scenario, not measured"
        output.append(state)
    result = pd.concat(output, ignore_index=True)
    first_high = result.loc[result.Risk_Level.eq("High")].groupby("Road_ID_Name").Year.min()
    result["First_High_Year"] = result.Road_ID_Name.map(first_high).astype("Int64")
    return result


def save_outputs(results, directory):
    folder = Path(directory)
    folder.mkdir(parents=True, exist_ok=True)
    cols = ["Scenario", "Year", "Road_ID_Name", "Road_Type", "Road_PCU_Volume", "Capacity_Limit", "VC_Ratio", "Risk_Level", "RF_Risk_Level", "RF_Disagreement",
"First_High_Year", "Capacity_Basis"]
    results[cols].to_csv(folder / "road_year_forecast.csv", index=False)
    results.loc[results.Year.eq(2035), ["Scenario", "Road_ID_Name", "Road_Type", "VC_Ratio", "Risk_Level", "First_High_Year"]].to_csv(folder / "scenario_2035_su
mmary.csv", index=False)


def compare_predefined_scenarios(history, model):
    """Return a small planning comparison for the predefined scenarios.

    The table is a sensitivity view of the stated assumptions. It is not a
    probability forecast or a confidence interval.
    """
    rows = []
    for name, parameters in PREDEFINED_SCENARIOS:
        result = simulate(history, model, name, **parameters)
        for year in (2025, 2030, 2035):
            frame = result.loc[result.Year.eq(year)]
            counts = frame.Risk_Level.value_counts().reindex(RISK_ORDER, fill_value=0)
            rows.append({
                "Scenario": name,
                "Year": year,
                "Low_Roads": int(counts["Low"]),
                "Medium_Roads": int(counts["Medium"]),
                "High_Roads": int(counts["High"]),
                "Max_VC": float(frame.VC_Ratio.max()),
            })
    return pd.DataFrame(rows)
```

```python
"""Interactive, terminal-only runner for the Metro Manila AADT model."""

from pathlib import Path
import re
import sys
import textwrap

import pandas as pd

ROOT = Path(__file__).resolve().parent
DATA = ROOT / "data" / "metro_manila_aadt_simulation_input_2020_2025.csv"


def ask_choice(prompt, choices):
    while True:
        answer = input(prompt).strip().lower()
        if answer in choices:
            return answer
        print(f"Please enter: {', '.join(sorted(choices))}")


def ask_year():
    while True:
        answer = input("Which year would you like to see? 2025-2035 [Enter = 2035]: ").strip()
        if not answer:
            return 2035
        if answer.isdigit() and 2025 <= int(answer) <= 2035:
            return int(answer)
        print("Enter a year from 2025 through 2035.")


def ask_growth():
    while True:
        answer = input("Yearly traffic growth percentage, 0-30 [Enter = 4.5]: ").strip()
        try:
            value = 4.5 if not answer else float(answer)
        except ValueError:
            value = -1
        if 0 <= value <= 30:
            return value / 100
        print("Enter a number from 0 to 30.")


def choose_scenario():
    print("\nCHOOSE A SCENARIO")
    print("-" * 100)
    print(f"{'No.':<5} {'Scenario':<31} {'What question does it answer?'}")
    print("-" * 100)
    rows = [
        ("1", "Baseline: 4.5% yearly growth", "What happens if traffic grows at the assumed rate?"),
        ("2", "Faster growth: 7% yearly", "What if traffic grows faster than expected?"),
        ("3", "Transit shift: 20% of cars", "What if some car users move to buses?"),
        ("4", "EDSA/C-5: 20% more capacity", "What if these roads gain capacity in 2028?"),
        ("5", "Your own growth rate", "What if traffic grows at a rate you choose?"),
```

```
        ("6", "Compare scenarios", "See the sensitivity range at 2025, 2030, and 2035."),
        ("0", "Exit", "Close the simulation."),
    ]
    for number, label, goal in rows:
        print(f"{number:<5} {label:<31} {goal}")
    print("-" * 100)
    choice = ask_choice("Enter a scenario number (0-6): ", set("0123456"))
    scenarios = {
        "1": ("Baseline 4.5%", {"growth_rate": 0.045}),
        "2": ("Accelerated 7%", {"growth_rate": 0.07}),
        "3": ("Transit shift 20%", {"growth_rate": 0.045, "transfer_share": 0.20}),
        "4": ("EDSA/C-5 expansion 20%", {"growth_rate": 0.045, "capacity_increase": 0.20}),
    }
    if choice == "5":
        growth = ask_growth()
        return f"Custom growth {growth * 100:g}%", {"growth_rate": growth}
    if choice == "6":
        return "__compare__", {}
    return scenarios.get(choice)


def show_comparison(comparison):
    width = 92
    border = "+" + "=" * (width - 2) + "+"
    print("\n" + border)
    print("| SCENARIO SENSITIVITY | planning comparison across stated assumptions".ljust(width - 1) + "|")
    print("| Changes in risk counts when assumptions change; this is not a probability forecast.".ljust(width - 1) + "|")
    print("|" + "-" * (width - 2) + "|")
    print("| " + f"{'Scenario':<25} {'Year':>4} {'Low':>5} {'Medium':>7} {'High':>6} {'Max V/C':>9}".ljust(width - 4) + " |")
    for _, row in comparison.iterrows():
        line = f"{row.Scenario:<25} {int(row.Year):>4} {int(row.Low_Roads):>5} {int(row.Medium_Roads):>7} {int(row.High_Roads):>6} {row.Max_VC:>8.1%}"
        print("| " + line.ljust(width - 4) + " |")
    print("|" + "-" * (width - 2) + "|")
    print("| Low <70%, Medium 70%-<95%, High >=95% of assumed capacity.".ljust(width - 1) + "|")
    print("| Max V/C = busiest road traffic divided by assumed capacity.".ljust(width - 1) + "|")
    print(border)


def show_results(result, year):
    frame = result.loc[result.Year.eq(year)].copy()
    baseline = result.loc[result.Year.eq(2025), ["Road_ID_Name", "Total_Raw_Volume_Reported", "Road_PCU_Volume"]].set_index("Road_ID_Name")
    counts = frame.Risk_Level.value_counts().reindex(["Low", "Medium", "High"], fill_value=0)
    year_alert = "High" if counts["High"] else "Medium" if counts["Medium"] else "Low"
    high_roads = frame.loc[frame.Risk_Level.eq("High"), "Road_ID_Name"].tolist()
    width = 108

    def border(char="-"):
        return "+" + char * (width - 2) + "+"

    def line(message=""):
        for part in textwrap.wrap(str(message), width - 4, break_long_words=False) or [""]:
            print("| " + part.ljust(width - 4) + " |")

    def table_line(message):
```

```
        if len(message) > width - 4:
            raise ValueError("Result row exceeds terminal box width")
        print("| " + message.ljust(width - 4) + " |")

    print("\n" + border("="))
    line(f"TRAFFIC RISK RESULT | {result.Scenario.iat[0]} | YEAR {year}")
    print(border())
    line(f"AT A GLANCE: {year} is {year_alert.upper()} ALERT for this 20-road group.")
    line(f"In {year}: {counts['High']} roads are High, {counts['Medium']} are Medium, and {counts['Low']} are Low.")
    line("The group alert uses the highest road risk; it does not mean every road has that risk.")
    line("High-risk roads this year: " + (", ".join(high_roads) if high_roads else "none"))
    print(border())
    line(f"ROAD-BY-ROAD RISK IN {year}")
    header = f"{'Road':<20} {'2025 vehicles':>13} {'2025 PCU':>10} {str(year) + ' PCU':>10} {'Cap. used':>9} {str(year) + ' risk':>9} {'AI risk':>8} {'High from':>9}"
    table_line(header)
    print(border())
    for _, road in frame.sort_values("VC_Ratio", ascending=False).iterrows():
        first_high = "-" if pd.isna(road.First_High_Year) else str(int(road.First_High_Year))
        row = (f"{road.Road_ID_Name:<20} {baseline.loc[road.Road_ID_Name, 'Total_Raw_Volume_Reported']:>13,.0f} "
               f"{baseline.loc[road.Road_ID_Name, 'Road_PCU_Volume']:>10,.0f} "
               f"{road.Road_PCU_Volume:>10,.0f} {road.VC_Ratio:>8.1%} "
               f"{road.Risk_Level:>9} {road.RF_Risk_Level:>8} {first_high:>9}")
        table_line(row)
    print(border())
    line("WHAT THE COLUMNS MEAN")
    line("2025 vehicles/day = reported vehicle count in the supplied dataset (source unverified).")
    line("2025 PCU/day = that vehicle mix converted to passenger-car units; this is calculated, not measured.")
    line(f"{year} PCU/day = projected daily traffic under the selected scenario. For 2025, this is the baseline, not a forecast.")
    line("Capacity used = projected traffic divided by assumed road capacity; the capacity is not measured.")
    line("Risk assessment = Low below 70%, Medium from 70% to below 95%, High at 95% or more.")
    line("AI assessment = Random Forest label. It can disagree with the threshold-based risk assessment.")
    line(f"The AI assessment differs from the threshold assessment for {int(frame.RF_Disagreement.sum())} roads in {year}.")
    line("High from = first modeled year at 95% capacity or more; '-' means no crossing by 2035.")
    line("These are scenario results for planning, not measured capacity or a prediction of physical road failure.")
    print(border("="))


def main():
    print("\nMETRO MANILA ROAD TRAFFIC SIMULATION | 2025-2035")
    print("Goal: explore when projected daily traffic may approach or exceed road capacity on 20 roads.")
    print("How to use: choose a scenario number, choose a year, then decide whether to save the full forecast.")
    print("After each result, the menu returns so you can compare another scenario. Enter 0 to exit.")
    print("The results are planning examples because the supplied road capacities are synthetic estimates.\n")
    if sys.version_info >= (3, 14):
        print("This CLI requires a supported scientific-Python runtime. Please recreate the environment with Python 3.12:")
        print("  py -3.12 -m venv .venv")
        print("  .\\.venv\\Scripts\\python -m pip install -r requirements-cli.txt")
        print("Then run this command again. Python 3.14 may block the SciPy DLL used by scikit-learn on Windows.\n")
        return
    print("Checking data and preparing the Random Forest risk model...")
    try:
        from aadt_model import add_pcu_and_risk, compare_predefined_scenarios, fit_and_evaluate, save_outputs, simulate, validate_data
    except (ImportError, OSError) as exc:
```

```
        print("\nThe scientific Python packages could not be loaded.")
        print("On Windows, recreate the environment with Python 3.12 and run the installation commands in README.md.")
        print(f"Technical detail: {exc}")
        return
    history = add_pcu_and_risk(validate_data(pd.read_csv(DATA)))
    model, metrics, _, comparison = fit_and_evaluate(history)
    print(f"Model check: the Random Forest matched {metrics.loc['accuracy', 'support']:.1%} of 2024-2025 risk labels.")
    print("This measures risk-label matching, not accuracy of future traffic forecasts.")
    while True:
        scenario = choose_scenario()
        if scenario is None:
            print("Goodbye.")
            return
        name, parameters = scenario
        if name == "__compare__":
            comparison = compare_predefined_scenarios(history, model)
            show_comparison(comparison)
            if ask_choice("Save this sensitivity table as CSV? (y/n): ", {"y", "n"}) == "y":
                destination = ROOT / "outputs" / "terminal" / "scenario_comparison.csv"
                comparison.to_csv(destination, index=False)
                print(f"Saved {destination}")
            continue
        result = simulate(history, model, name, **parameters)
        year = ask_year()
        show_results(result, year)
        if ask_choice("Save every year of this scenario as CSV files? (y/n): ", {"y", "n"}) == "y":
            slug = re.sub(r"[^a-z0-9]+", "_", name.lower()).strip("_")
            destination = ROOT / "outputs" / "terminal" / slug
            save_outputs(result, destination)
            print(f"Saved {destination / 'road_year_forecast.csv'}")
            print(f"Saved {destination / 'scenario_2035_summary.csv'}")


if __name__ == "__main__":
    try:
        main()
    except (KeyboardInterrupt, EOFError):
        print("\nStopped.")
```